

**SIMATS ENGINEERING
ASSESSMENT TOOL 1****COURSE NAME: Database Management Systems****COURSE CODE: CSA05**

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I DARA KUSUMANJALI(192425398L) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

STUDENT_COURSE Relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Functional Dependencies:

1. $SID \rightarrow SName$
Student ID uniquely determines the Student Name.
2. $CID \rightarrow CName, Instructor$
Course ID uniquely determines the Course Name and Instructor.
3. $(SID, CID) \rightarrow Grade$
The combination of Student ID and Course ID determines the Grade.

Candidate Key:

The closure of (SID, CID) is:

$(SID, CID)^+ = \{SID, SName, CID, CName, Instructor, Grade\}$

Since all attributes are obtained, (SID, CID) is a superkey.

SID alone cannot determine all attributes.

CID alone cannot determine all attributes.

Therefore, the minimal key is:

Candidate Key = (SID, CID)

Conclusion:

The functional dependencies are:

$SID \rightarrow SName$

$CID \rightarrow CName, Instructor$

$(SID, CID) \rightarrow Grade$

The only candidate key of the relation is **(SID, CID)**.

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Conversion of an Unnormalized Relation (UNF) into First Normal Form (1NF)

1. Unnormalized Form (UNF)

Consider the following relation:

STUDENT(SID, SName, Courses)

SID	SName	Courses
S101	Arun	DBMS, Java
S102	Priya	Python, DBMS

Here, the **Courses** attribute contains multiple values in a single cell. Therefore, the relation is in **UNF** because it contains a repeating group or multivalued attribute.

2. Identify the Repeating Group

The attribute **Courses** contains more than one value for a student.

For example:

$S101 \rightarrow DBMS, Java$

$S102 \rightarrow Python, DBMS$

This violates the requirement of 1NF that every attribute must contain a **single, atomic value**.

3. Remove the Repeating Group

To convert the relation into 1NF, each course value is placed in a separate row.

The relation becomes:

STUDENT(SID, SName, Course)

SID	SName	Course
S101	Arun	DBMS
S101	Arun	Java
S102	Priya	Python
S102	Priya	DBMS

Now, every cell contains only one value.

4. Identify the Primary Key

Since a student can take multiple courses, **SID alone cannot uniquely identify each row.**

Therefore, the combination of **SID and Course** can be used as the primary key.

Primary Key = **(SID, Course)**

5. Step-by-Step Justification

UNF:

STUDENT(SID, SName, {Courses})

The braces indicate a repeating group.

Remove the repeating group:

STUDENT(SID, SName, Course)

Ensure that every attribute contains only one value:

$SID \rightarrow \text{Single value}$

$SName \rightarrow \text{Single value}$

$Course \rightarrow \text{Single value}$

Therefore, the relation satisfies the atomicity requirement of 1NF.

Final 1NF Relation:

STUDENT(SID, SName, Course)

Primary Key = (SID, Course)

Conclusion:

An unnormalized relation is converted into **1NF** by removing repeating groups and ensuring that every attribute contains only **atomic (single) values**. In the example, the multiple courses stored in one cell are separated into individual rows, making the relation conform to **First Normal Form (1NF)**.

3. Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$.

Analyze and decompose it to 2NF eliminating partial dependencies.

Given Relation:

R(A, B, C, D, E)

Functional Dependencies:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

Step 1: Find the Candidate Key

Starting with A:

$A \rightarrow BC$

Therefore, we get B and C.

Using $BC \rightarrow D$:

$BC \rightarrow D$

Therefore, we get D.

Using $D \rightarrow E$:

$D \rightarrow E$

Therefore, we get E.

Hence:

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes of R, **A is a candidate key.**

Candidate Key = A

Step 2: Check for Partial Dependencies

A relation is in 2NF when:

- 1.

It is already in 1NF.

- 2.

- 3.

No non-prime attribute is partially dependent on a candidate key.

- 4.

Here, the candidate key consists of only **one attribute**, A.

Therefore, a partial dependency cannot exist because partial dependency requires a **composite candidate key**.

The dependency:

$A \rightarrow B, C$

is a **full functional dependency**, not a partial dependency.

Therefore, **R is already in 2NF**.

Step 3: Important Observation

Although there are dependencies:

$BC \rightarrow D$

$D \rightarrow E$

these are not partial dependencies on a part of the candidate key A.

They represent dependencies between non-key attributes and are relevant when checking **3NF**, not 2NF.

Final Result:

Original Relation:

$R(A, B, C, D, E)$

Candidate Key:

A

Partial Dependencies:

None

Therefore:

R is already in 2NF, so no decomposition is required to achieve 2NF.

However, the relation is **not in 3NF** because:

$BC \rightarrow D$ and $D \rightarrow E$ involve non-prime attributes determining other non-prime attributes. A further decomposition would be required for 3NF.

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

3NF Decomposition of EMPLOYEE Relation

Given Relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Functional Dependencies:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore, there is a transitive dependency:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Here, **EmpID** is the candidate key.

Step 1: Identify the Transitive Dependency

From the functional dependencies:

$\text{EmpID} \rightarrow \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore, by transitivity:

$\text{EmpID} \rightarrow \text{DeptName}, \text{ManagerID}$

This means that non-key attribute **DeptID** determines other non-key attributes **DeptName** and **ManagerID**.

This violates **Third Normal Form (3NF)** because a non-key attribute is determining non-key attributes.

Step 2: Decompose the Relation

To remove the transitive dependency, divide the relation into two relations.

Relation 1: EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Functional dependency:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

Primary Key: **EmpID**

Relation 2: DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Functional dependency:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Primary Key: **DeptID**

Step 3: Verify the Decomposition

In EMPLOYEE:

EmpID is the primary key and determines all other attributes.

In DEPARTMENT:

DeptID is the primary key and determines DeptName and ManagerID.

Thus, there is no transitive dependency within either relation.

Final 3NF Decomposition:

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

Primary Keys:

EMPLOYEE $\rightarrow$ EmpID

DEPARTMENT $\rightarrow$ DeptID

Foreign Key:

EMPLOYEE.DeptID references DEPARTMENT.DeptID.

Conclusion:

The original EMPLOYEE relation contains the transitive dependency:

EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName, ManagerID

By decomposing it into **EMPLOYEE(EmpID, EmpName, DeptID)** and

DEPARTMENT(DeptID, DeptName, ManagerID), the transitive dependency is eliminated and the relations satisfy 3NF.

5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

BCNF Analysis of $R(A, B, C, D)$

Given Relation:

$R(A, B, C, D)$

Functional Dependencies:

$AB \rightarrow C$

$C \rightarrow D$

$D \rightarrow A$

Step 1: Find the Candidate Keys

First, consider **AB**:

$AB \rightarrow C$

$C \rightarrow D$

Therefore:

$AB^+ = \{A, B, C, D\}$

So, **AB is a candidate key.**

Now consider **BC**:

$C \rightarrow D$

$D \rightarrow A$

Therefore:

$BC^+ = \{A, B, C, D\}$

So, **BC** is also a candidate key.

Now consider **BD**:

$D \rightarrow A$

Using A and B:

$AB \rightarrow C$

Therefore:

$BD^+ = \{A, B, C, D\}$

So, **BD** is also a candidate key.

Therefore, the candidate keys are:

AB, BC, BD

Step 2: Check BCNF Condition

A relation is in **BCNF** if, for every non-trivial functional dependency $X \rightarrow Y$, **X must be a superkey**.

Check each FD:

1. $AB \rightarrow C$

AB is a candidate key, so AB is a superkey.

Therefore, this dependency satisfies BCNF.

2. $C \rightarrow D$

C alone does not determine all attributes.

$C^+ = \{C, D, A\}$

It does not contain B.

Therefore, C is **not a superkey**.

Hence, $C \rightarrow D$ violates BCNF.

3. $D \rightarrow A$

D alone does not determine all attributes.

$D^+ = \{D, A\}$

It does not contain B or C.

Therefore, D is **not a superkey**.

Hence, $D \rightarrow A$ also violates BCNF.

Therefore, **R is not in BCNF**.

Step 3: Decompose Using $C \rightarrow D$

Since $C \rightarrow D$ violates BCNF, decompose R into:

$R_1(C, D)$

$R_2(A, B, C)$

$R_1(C, D)$:

FD: $C \rightarrow D$

C is a candidate key of R_1 .

Therefore, R_1 is in BCNF.

$R_2(A, B, C)$:

The relevant dependency is:

$AB \rightarrow C$

AB is a candidate key of R2.

Therefore, R2 is also in BCNF.

Final BCNF Decomposition:

R1(C, D)

R2(A, B, C)

Conclusion:

The original relation **R(A, B, C, D)** is **not in BCNF** because **$C \rightarrow D$** and **$D \rightarrow A$** have determinants that are not superkeys.

After decomposition:

R1(C, D)

R2(A, B, C)

both relations satisfy **BCNF**.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Lossless-Join Decomposition Using the Chase Algorithm

Given Relation:

Consider the relation:

R(A, B, C)

with the functional dependency:

$A \rightarrow B$

Decompose R into:

R1(A, B)

R2(A, C)

The objective is to experimentally verify whether this decomposition is **lossless-join** using the **tabular (Chase) algorithm**.

Step 1: Create the Chase Table

Create one row for each decomposed relation and one column for each attribute.

For the attributes A, B, C, assign distinct symbols initially.

Relation	A	B	C
R1 (A, B)	a	b	c1
R2 (A, C)	a	b2	c

Here, common attributes receive the same symbol. Therefore, A in both rows is represented by the same symbol **a**.

Step 2: Apply the Functional Dependency

Given:

$A \rightarrow B$

Since both rows have the same value **a** for A, the functional dependency requires their B values to be equal.

Therefore, replace **b2** with **b**.

The table becomes:

Relation	A	B	C
R1 (A, B)	a	b	c1
R2 (A, C)	a	b	c

Step 3: Check the Result

After applying the functional dependency, the second row becomes:

(a, b, c)

where all three values correspond to the distinguished symbols of the original attributes.

Therefore, the chase succeeds.

Step 4: Conclusion

Since a row containing all distinguished symbols **(a, b, c)** is obtained, the decomposition:

$R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)$

is **lossless-join**.

Therefore:

$R_1 \bowtie R_2 = R$

and no spurious tuples are generated after joining the decomposed relations.

Final Answer:

The Chase algorithm produces a row containing all original attribute symbols:

(a, b, c)

Hence, the decomposition is **lossless-join**.

7, Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

4NF Decomposition of TEACHER Relation

Given Relation:

TEACHER(TID, Subject, Hobby)

Assume that each teacher can teach multiple subjects and can have multiple hobbies, and these two facts are independent of each other.

Step 1: Identify the Multi-Valued Dependencies

For a teacher, the set of subjects is independent of the set of hobbies.

Therefore, the multi-valued dependencies are:

TID $\twoheadrightarrow$ Subject

TID $\twoheadrightarrow$ Hobby

Here, $\twoheadrightarrow$ represents a multi-valued dependency (MVD).

For example, if a teacher has:

TID = T01

Subjects = DBMS, Java

Hobbies = Music, Cricket

the relation may contain all combinations:

T01, DBMS, Music

T01, DBMS, Cricket

T01, Java, Music

T01, Java, Cricket

This creates unnecessary redundancy.

Step 2: Check for 4NF Violation

A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial multi-valued dependency $X \twoheadrightarrow Y$, X must be a superkey.

Here:

TID $\rightarrow\rightarrow$ Subject

But TID alone does not determine a single tuple because a teacher can have multiple subjects.

Similarly:

TID $\rightarrow\rightarrow$ Hobby

TID is not a superkey of the original relation.

Therefore, the relation **TEACHER(TID, Subject, Hobby)** is not in 4NF.

Step 3: Decompose the Relation

To eliminate the multi-valued dependencies, decompose TEACHER into two relations.

Relation 1: TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

MVD:

TID $\rightarrow\rightarrow$ Subject

Primary Key:

(TID, Subject)

Relation 2: TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

MVD:

TID $\rightarrow\rightarrow$ Hobby

Primary Key:

(TID, Hobby)

Step 4: Verify 4NF

In TEACHER_SUBJECT, the attributes are only TID and Subject, so there is no independent multi-valued attribute.

In TEACHER_HOBBY, the attributes are only TID and Hobby, so there is no independent multi-valued attribute.

Therefore, both relations satisfy **4NF**.

Final 4NF Decomposition:

TEACHER_SUBJECT(TID, Subject)

TEACHER_HOBBY(TID, Hobby)

Conclusion:

The original relation contains the multi-valued dependencies:

TID $\twoheadrightarrow$ Subject

TID $\twoheadrightarrow$ Hobby

Since these independent multi-valued dependencies violate 4NF, the relation is decomposed into:

TEACHER_SUBJECT(TID, Subject)

TEACHER_HOBBY(TID, Hobby)

Thus, the decomposition eliminates redundancy and produces relations in **Fourth Normal Form (4NF)**.

**8. Demonstrate how join dependencies lead to redundancy.
Decompose a given relation to 5NF (Project-Join Normal Form).**

5NF Decomposition Using Join Dependency

Given Relation:

Consider the relation:

SUPPLY(Supplier, Part, Project)

Assume that a supplier can supply a particular part to a project. The relationship among **Supplier, Part, and Project** is represented in one relation.

For example:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Here, the relationship can be represented using three independent pairwise relationships:

Supplier-Part

Supplier-Project

Part-Project

Step 1: Identify the Join Dependency

The relation has the join dependency:

(Supplier, Part), (Supplier, Project), (Part, Project)

This means that the original SUPPLY relation can be reconstructed by joining these three projections.

Therefore:

$$\text{SUPPLY} = \pi(\text{Supplier, Part})(\text{SUPPLY}) \bowtie \pi(\text{Supplier, Project})(\text{SUPPLY}) \bowtie \pi(\text{Part, Project})(\text{SUPPLY})$$

Step 2: Demonstrate Redundancy

Suppose the following information exists:

Supplier-Part:

S1, P1

S1, P2

Supplier-Project:

S1, J1

S1, J2

Part-Project:

P1, J1

P1, J2

P2, J1

P2, J2

When these relations are joined, all valid combinations are generated:

S1, P1, J1

S1, P1, J2

S1, P2, J1

S1, P2, J2

If the same information is stored directly in one three-attribute relation, repeated combinations can create unnecessary redundancy.

For example, supplier S1 is repeated for every combination of part and project.

Step 3: Decompose the Relation

To remove the redundancy caused by the join dependency, decompose SUPPLY into three relations:

SUPPLIER_PART(Supplier, Part)

SUPPLIER_PROJECT(Supplier, Project)

PART_PROJECT(Part, Project)

Each relation stores only one type of relationship.

Step 4: Verify the 5NF Condition

A relation is in **5NF (Project-Join Normal Form)** if every non-trivial join dependency is implied by the candidate keys.

The decomposition separates the independent relationships and eliminates the non-trivial join dependency from the original relation.

Therefore, the decomposed relations satisfy **5NF**.

Final 5NF Decomposition:

SUPPLIER_PART(Supplier, Part)

SUPPLIER_PROJECT(Supplier, Project)

PART_PROJECT(Part, Project)

Conclusion:

A join dependency can cause redundancy when several independent relationships are stored together in a single relation. In the SUPPLY relation, supplier-part, supplier-project, and part-project relationships are independent. Decomposing the relation into three smaller relations eliminates the redundancy and gives a **lossless decomposition into 5NF (Project-Join Normal Form)**.

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Normalization of a Poorly Designed College Database Schema to 3NF

Given Relation:

Consider the following poorly designed relation:

COLLEGE_STUDENT(SID, SName, DeptID, DeptName, CourseID, CourseName, Instructor, Grade)

This relation stores student, department, course, instructor, and grade information in a single table.

Functional Dependencies:

$SID \rightarrow SName, DeptID$

$DeptID \rightarrow DeptName$

$CourseID \rightarrow CourseName, Instructor$

$(SID, CourseID) \rightarrow Grade$

Therefore, there are transitive dependencies:

$SID \rightarrow DeptID \rightarrow DeptName$

and

$(SID, CourseID) \rightarrow CourseID \rightarrow CourseName, Instructor$

1. Insertion Anomaly

Suppose a new department is created but no student has been admitted to that department yet.

In the original table, it is difficult to store the department information because SID and other student-related information may be required.

Similarly, a new course cannot be stored unless a student is registered for it.

Therefore, the database has an **insertion anomaly**.

2. Deletion Anomaly

Suppose the only student enrolled in a particular course is deleted.

If the student's complete record is deleted, the information about the course and its instructor may also be lost.

Similarly, if the only student belonging to a department is deleted, the department information may also disappear.

Therefore, the database has a **deletion anomaly**.

3. Update Anomaly

Suppose the name of a department changes.

The department name may appear in many student records. Therefore, the same value must be updated in multiple rows.

If some rows are updated and others are not, inconsistent department information will exist.

Similarly, if an instructor's information changes, multiple records may need to be updated.

Therefore, the database has an **update anomaly**.

4. First Normal Form (1NF)

The relation is assumed to contain only atomic values, with no repeating groups.

Therefore:

COLLEGE_STUDENT(SID, SName, DeptID, DeptName, CourseID, CourseName, Instructor, Grade)

is in 1NF.

5. Convert to 2NF

The candidate key is:

(SID, CourseID)

However:

$SID \rightarrow SName, DeptID$

This is a partial dependency because SName and DeptID depend only on part of the composite key.

Also:

$CourseID \rightarrow CourseName, Instructor$

This is another partial dependency.

Therefore, decompose the relation into:

STUDENT(SID, SName, DeptID)

COURSE(CourseID, CourseName, Instructor)

ENROLLMENT(SID, CourseID, Grade)

Now, partial dependencies are removed.

6. Convert to 3NF

In the STUDENT relation:

$SID \rightarrow DeptID$

$DeptID \rightarrow DeptName$

Therefore:

$SID \rightarrow DeptID \rightarrow DeptName$

This is a transitive dependency.

To remove it, create a separate DEPARTMENT relation.

The final relations are:

STUDENT(SID, SName, DeptID)

Primary Key: SID

Foreign Key: DeptID

DEPARTMENT(DeptID, DeptName)

Primary Key: DeptID

COURSE(CourseID, CourseName, Instructor)

Primary Key: CourseID

ENROLLMENT(SID, CourseID, Grade)

Primary Key: (SID, CourseID)

Foreign Keys: SID references STUDENT and CourseID references COURSE.

7. Final 3NF Schema

STUDENT(SID, SName, DeptID)

DEPARTMENT(DeptID, DeptName)

COURSE(CourseID, CourseName, Instructor)

ENROLLMENT(SID, CourseID, Grade)

8. Verification of 3NF

In STUDENT:

$SID \rightarrow SName, DeptID$

SID is the key, so the relation satisfies 3NF.

In DEPARTMENT:

$DeptID \rightarrow DeptName$

DeptID is the key, so the relation satisfies 3NF.

In COURSE:

$CourseID \rightarrow CourseName, Instructor$

CourseID is the key, so the relation satisfies 3NF.

In ENROLLMENT:

$(SID, CourseID) \rightarrow Grade$

The complete composite key determines Grade, so the relation satisfies 3NF.

Conclusion:

The original college relation suffers from **insertion, deletion, and update anomalies** because student, department, course, and enrollment information are stored together. By decomposing it into **STUDENT, DEPARTMENT, COURSE, and ENROLLMENT**, partial and transitive dependencies are eliminated. The resulting database is in **Third Normal Form (3NF)** and has reduced redundancy and improved data consistency.

10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Attribute Closure and Candidate Keys

Consider the relation:

R(A, B, C, D, E)

with the following functional dependencies:

$A \rightarrow B$

$B \rightarrow C$

$CD \rightarrow E$

$E \rightarrow A$

The objective is to compute attribute closures and determine all candidate keys.

1. Closure of A

Start with:

$A^+ = \{A\}$

Using $A \rightarrow B$:

$A^+ = \{A, B\}$

Using $B \rightarrow C$:

$A^+ = \{A, B, C\}$

There is no D, so $CD \rightarrow E$ cannot be applied.

Therefore:

$A^+ = \{A, B, C\}$

Since A^+ does not contain all attributes, **A is not a candidate key.**

2. Closure of B

Start with:

$$\mathbf{B^+ = \{B\}}$$

Using $B \rightarrow C$:

$$\mathbf{B^+ = \{B, C\}}$$

No other dependency can be applied.

Therefore:

$$\mathbf{B^+ = \{B, C\}}$$

So, B is not a candidate key.

3. Closure of C

Start with:

$$\mathbf{C^+ = \{C\}}$$

No dependency can be applied.

Therefore:

$$\mathbf{C^+ = \{C\}}$$

So, C is not a candidate key.

4. Closure of D

Start with:

$$\mathbf{D^+ = \{D\}}$$

No dependency can be applied.

Therefore:

$$\mathbf{D^+ = \{D\}}$$

So, **D is not a candidate key.**

5. Closure of E

Start with:

$$\mathbf{E^+ = \{E\}}$$

Using $E \rightarrow A$:

$$\mathbf{E^+ = \{E, A\}}$$

Using $A \rightarrow B$:

$$\mathbf{E^+ = \{E, A, B\}}$$

Using $B \rightarrow C$:

$$\mathbf{E^+ = \{E, A, B, C\}}$$

D is still missing.

Therefore:

$$\mathbf{E^+ = \{E, A, B, C\}}$$

So, **E is not a candidate key.**

6. Find Candidate Keys

Since **D does not appear on the right-hand side of any functional dependency**, D cannot be derived from other attributes.

Therefore, **every candidate key must contain D.**

Consider **AD**:

$$AD^+ = \{A, D\}$$

$A \rightarrow B$ gives B.

$B \rightarrow C$ gives C.

$CD \rightarrow E$ gives E.

Therefore:

$$AD^+ = \{A, B, C, D, E\}$$

Hence, **AD is a candidate key.**

Consider **BD**:

$$BD^+ = \{B, D\}$$

$B \rightarrow C$ gives C.

$CD \rightarrow E$ gives E.

$E \rightarrow A$ gives A.

Therefore:

$$BD^+ = \{A, B, C, D, E\}$$

Hence, **BD is a candidate key.**

Consider **DE**:

$$DE^+ = \{D, E\}$$

$E \rightarrow A$ gives A.

$A \rightarrow B$ gives B.

$B \rightarrow C$ gives C.

Therefore:

$DE^+ = \{A, B, C, D, E\}$

Hence, **DE is a candidate key.**

Final Answer

Candidate Keys:

AD, BD, DE

All three are minimal because removing any attribute from them prevents the closure from containing all attributes.

Therefore, the relation has three candidate keys: AD, BD, and DE.